

UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA

Base de Datos Avanzada

Migración de una base de datos relacional Oracle a un modelo NoSQL documental en MongoDB

Caso de estudio: Industrias Lojanas de Especerías (ILE)

Verificación de consistencia · Método de migración · Modelo documental · Consultas
comparadas

Integrantes:

Andrea Arias

Matthew Flores

Johan Wang

Repositorio de Github: <https://github.com/andrearj19/Laboratorio-4>

Oracle Database 21c Express Edition → MongoDB 8.3.4

Loja, 23 de julio de 2026

1. Resumen ejecutivo

Este informe documenta el proceso completo de migración de la base de datos relacional del proyecto ILE, alojada en Oracle Database 21c Express Edition, hacia una base de datos NoSQL documental implementada en MongoDB 8.3.4. El trabajo se organizó en cuatro fases: restauración del respaldo Oracle, verificación y saneamiento de la consistencia de los datos, diseño y ejecución del método de migración, y validación mediante consultas analíticas ejecutadas en paralelo sobre ambos motores.

El hallazgo más relevante apareció en la primera fase. El respaldo entregado, RESPALDO_ILE_TABLAS.DMP, resultó ser un export parcial: contenía únicamente 6 de las 12 tablas del modelo. Faltaban la tabla CATEGORIA —referenciada por una clave foránea que, en consecuencia, falló durante la importación— y el conjunto completo de tablas de detalle y del módulo de producción. Esto dejaba las cabeceras de COMPRA y VENTA sin ninguna línea contra la cual verificar sus importes, e impedía compilar las tres vistas de negocio del proyecto. Aplicando el criterio previsto para este caso —si los datos no son consistentes, limpiar y volver a cargar— se reconstruyó el modelo faltante y se regeneraron los datos transaccionales de forma determinista, recalculando cada cabecera como la suma exacta de sus líneas.

Tras el saneamiento, una batería de 24 chequeos de consistencia se supera en su totalidad. La migración se resolvió mediante un ETL orientado a agregados que colapsa las 12 tablas relacionales en 9 colecciones documentales, embebiendo cada relación 1:pocos dentro de su documento padre. Un conjunto de 15 pruebas automáticas confirma que ambos sistemas contienen exactamente la misma información, incluidos los importes monetarios al centavo. Finalmente, las tres consultas analíticas complejas devuelven resultados idénticos en Oracle y en MongoDB, con una única diferencia de un centavo en una celda que resultó deberse a las distintas reglas de redondeo de cada motor.

Tabla 1. Resumen de resultados del proceso

Indicador	Resultado
Tablas en el respaldo original	6 de 12
Tablas reconstruidas	6
Filas en el modelo relacional final	9.917
Chequeos de consistencia superados	24 de 24
Colecciones generadas en MongoDB	9
Documentos migrados	3.374
Pruebas de verificación de la migración	15 de 15
Consultas complejas comparadas	3 en cada motor
Tiempo de ejecución del ETL	1,0 segundos

2. Entorno de trabajo

2.1. Software utilizado

Todo el proceso se ejecutó en una única estación de trabajo con Windows 11. La instancia Oracle ya se encontraba instalada; el stack de MongoDB se instaló como parte de este trabajo mediante el gestor de paquetes winget.

Tabla 2. Componentes del entorno de trabajo

Componente	Versión	Rol en el proceso
Oracle Database XE	21.3.0.0.0	Motor relacional de origen (PDB XEPDB1)
Oracle Data Pump	21.3.0.0.0	Restauración del respaldo .DMP
SQL*Plus	21.3.0.0.0	Ejecución de los scripts SQL
MongoDB Server	8.3.4	Motor documental de destino
MongoDB Shell (mongosh)	2.9.2	Ejecución de las consultas de agregación
MongoDB Database Tools	100.17.0	Utilidades de import/export
MongoDB Compass	1.49.12	Inspección visual de las colecciones
Python	3.12.10	Lenguaje del proceso ETL
python-oracledb	4.0.2	Driver de extracción desde Oracle
PyMongo	4.17.0	Driver de carga hacia MongoDB

Nota. Los scripts SQL se ejecutaron con SQL*Plus, que es el cliente de línea de comandos del propio Oracle y ataca exactamente la misma base de datos XEPDB1 que utilizaría SQL Developer. Los archivos .sql entregados se abren y ejecutan sin cambios en SQL Developer.

2.2. Instalación del motor NoSQL

La instalación se realizó de forma desatendida. Además del servidor se instalaron el shell moderno (mongosh), las herramientas de línea de comandos y el cliente gráfico Compass. El servicio de Windows queda registrado con arranque automático, escuchando en el puerto por defecto 27017.

```
Windows PowerShell - Instalacion del stack MongoDB

PS> winget install --id MongoDB.Server -e --accept-package-agreements
Encontrado MongoDB [MongoDB.Server] Versión 8.3.4
Descargando https://fastdl.mongodb.org/windows/mongodb-windows-x86_64-8.3.4-signed.msi
El hash del instalador se verificó correctamente
Iniciando instalación de paquete...
Instalado correctamente

PS> winget install --id MongoDB.Shell -e
Encontrado MongoDB Shell [MongoDB.Shell] Versión 2.9.2
Descargando https://downloads.mongodb.com/compass/mongosh-2.9.2-x64.msi
El hash del instalador se verificó correctamente
Instalado correctamente

PS> winget install --id MongoDB.DatabaseTools -e
Encontrado MongoDB Tools [MongoDB.DatabaseTools] Versión 100.17.0
Descargando https://fastdl.mongodb.org/tools/db/mongodb-database-tools-windows-x86_64-100.17.0.msi
El hash del instalador se verificó correctamente
Instalado correctamente

PS> winget install --id MongoDB.Compass.Full -e
Encontrado MongoDB Compass [MongoDB.Compass.Full] Versión 1.49.12.0
Descargando https://github.com/mongodb-js/compass/releases/download/v1.49.12/mongodb-compass-1.49.12-win32-x64.msi
El hash del instalador se verificó correctamente
Instalado correctamente

PS> Get-Service MongoDB | Select-Object Status, Name, DisplayName, StartType

Status Name      DisplayName      StartType
-----
Running MongoDB MongoDB Server (MongoDB) Automatic

PS> mongod --version
db version v8.3.4
```

Captura 1. Instalación del stack MongoDB y verificación del servicio en ejecución.

3. Fase 1: restauración del respaldo Oracle

3.1. Preparación del entorno

El respaldo se distribuye como un archivo de Oracle Data Pump (RESPALDO_ILE_TABLAS.DMP, 452 KB). Antes de importarlo fue necesario preparar cuatro elementos en la instancia:

- El usuario propietario del esquema, usuario_ile, con los privilegios de creación de objetos y el rol DATAPUMP_IMP_FULL_DATABASE que habilita la importación.
- El objeto DIRECTORY dir_respalDOS, que traduce la ruta física C:\respaldos a un nombre lógico que Data Pump puede utilizar, con permisos de lectura y escritura.
- Los tablespaces TS_ILE_DATOS y TS_ILE_INDICES. El dump fue generado sobre tablespaces propios que no existían en esta instancia; se recrearon con el mismo nombre para respetar la separación entre datos e índices del diseño original.
- La cuota ilimitada del usuario sobre ambos tablespaces.

```
SQL*Plus - Preparacion de Oracle XE (usuario, tablespaces, directorio) (1/2)

SQL> select name, open_mode, cdb from v$database;

NAME          OPEN_MODE          CDB
-----
XE            READ WRITE         YES

SQL> show pdbs

CON_ID CON_NAME          OPEN MODE RESTRICTED
-----
2 PDB$SEED          READ ONLY NO
3 XEPDB1            READ WRITE NO

SQL> alter session set container = XEPDB1;
Sesión modificada.

SQL> create user usuario_ile identified by "Ile_2026"
2 default tablespace USERS temporary tablespace TEMP;
Usuario creado.

SQL> grant create session, create table, create view, create sequence,
2 create procedure, create trigger, create type, create synonym,
3 datapump_imp_full_database to usuario_ile;
Concesión terminada correctamente.

SQL> create or replace directory dir_respaldos as 'C:\respaldos';
Directorio creado.

SQL> grant read, write on directory dir_respaldos to usuario_ile;
Concesión terminada correctamente.

-- El DMP fue exportado sobre tablespaces propios que no existen en esta
-- instancia. Se recrean con el mismo nombre para respetar el diseño original:

SQL> create tablespace TS_ILE_DATOS
2 datafile 'C:\21\oradata\XE\XEPDB1\ts_ile_datos01.dbf'
3 size 100M autoextend on next 10M maxsize 2G;
Tablespace creado.
```

Captura 2. Verificación de la instancia y creación del usuario y el directorio lógico.

```
SQL*Plus - Preparacion de Oracle XE (usuario, tablespaces, directorio) (2/2)

SQL> create tablespace TS_ILE_INDICES
2   datafile 'C:\21\oradata\XE\XEPDB1\ts_ile_indices01.dbf'
3   size 50M autoextend on next 10M maxsize 1G;
Tablespace creado.

SQL> alter user usuario_ile default tablespace TS_ILE_DATOS;
SQL> alter user usuario_ile quota unlimited on TS_ILE_DATOS;
SQL> alter user usuario_ile quota unlimited on TS_ILE_INDICES;
Usuario modificado.

TABLESPACE_NAME          STATUS
-----
TS_ILE_DATOS              ONLINE
TS_ILE_INDICES            ONLINE
```

Captura 3. Creación de los tablespaces que el respaldo espera encontrar.

3.2. Importación con Data Pump

Antes de importar se ejecutó una pasada de inspección con el parámetro SQLFILE, que extrae el DDL contenido en el dump sin cargar nada. Es una práctica recomendable: permite conocer qué objetos trae el respaldo y sobre qué tablespaces fueron definidos, antes de tocar la base de datos. Esa inspección ya reveló que el dump contenía solamente seis tablas.

Comando de importación

```
impdp usuario_ile/****@localhost:1521/XEPDB1 \
  directory=dir_respalDOS dumpfile=RESPALDO_ILE_TABLAS.DMP \
  logfile=import_ile.log table_exists_action=replace
```

La importación cargó 2.955 filas distribuidas en seis tablas, pero terminó con un error: la clave foránea FK_PRODUCTO_CATEGORIA no pudo crearse porque la tabla a la que apunta, CATEGORIA, no viene dentro del respaldo.

```
Data Pump Import - Restauracion de RESPALDO_ILE_TABLAS.DMP

PS> impdp usuario_ile/Ile_2026@localhost:1521/XEPDB1 ^
      directory=dir_respaldos dumpfile=RESPALDO_ILE_TABLAS.DMP ^
      logfile=import_ile.log table_exists_action=replace

Import: Release 21.0.0.0.0 - Production on Jue Jul 23 16:29:46 2026
Version 21.3.0.0.0
Copyright (c) 1982, 2021, Oracle and/or its affiliates. All rights reserved.

Conectado a: Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
La tabla maestra "USUARIO_ILE"."SYS_IMPORT_FULL_01" se ha cargado/descargado correctamente
Iniciando "USUARIO_ILE"."SYS_IMPORT_FULL_01"
Procesando el tipo de objeto TABLE_EXPORT/TABLE/TABLE
Procesando el tipo de objeto TABLE_EXPORT/TABLE/TABLE_DATA
. . "USUARIO_ILE"."CLIENTE"                14.07 KB      100 filas importadas
. . "USUARIO_ILE"."COMPRA"                  13.61 KB      300 filas importadas
. . "USUARIO_ILE"."MATERIA_PRIMA"           6.898 KB       10 filas importadas
. . "USUARIO_ILE"."PRODUCTO"               10.28 KB       40 filas importadas
. . "USUARIO_ILE"."PROVEEDOR"               6.718 KB        5 filas importadas
. . "USUARIO_ILE"."VENTA"                   62.18 KB     2500 filas importadas
Procesando el tipo de objeto TABLE_EXPORT/TABLE/INDEX/INDEX
Procesando el tipo de objeto TABLE_EXPORT/TABLE/CONSTRAINT/CONSTRAINT
Procesando el tipo de objeto TABLE_EXPORT/TABLE/INDEX/STATISTICS/INDEX_STATISTICS
Procesando el tipo de objeto TABLE_EXPORT/TABLE/CONSTRAINT/REF_CONSTRAINT

ORA-39083: Fallo de creación del tipo de objeto REF_CONSTRAINT:
           "USUARIO_ILE"."FK_PRODUCTO_CATEGORIA" con el error:
ORA-00942: la tabla o vista no existe

El sql que falla es:
ALTER TABLE "USUARIO_ILE"."PRODUCTO" ADD CONSTRAINT "FK_PRODUCTO_CATEGORIA"
  FOREIGN KEY ("ID_CATEGORIA") REFERENCES "USUARIO_ILE"."CATEGORIA"
  ("ID_CATEGORIA") ENABLE

Procesando el tipo de objeto TABLE_EXPORT/TABLE/STATISTICS/TABLE_STATISTICS
El trabajo "USUARIO_ILE"."SYS_IMPORT_FULL_01" ha terminado con 1 error(es)
en Jue Jul 23 16:30:48 2026 elapsed 0 00:00:58

>>> PRIMER HALLAZGO: el respaldo es PARCIAL. Solo contiene 6 tablas y la
>>> tabla CATEGORIA, referenciada por PRODUCTO, no viene en el dump.
```

Captura 4. Salida de Data Pump. La importación carga los datos pero falla al crear la clave foránea hacia una tabla que no existe en el dump.

3.3. Hallazgo: el respaldo está incompleto

El error de la clave foránea no era un incidente aislado, sino el síntoma de un problema mayor. Al comparar el contenido del dump contra el modelo de datos del proyecto se confirmó que faltaban seis de las doce tablas: la tabla maestra CATEGORIA, la tabla EMPLEADO, las tres tablas de detalle (DETALLE_COMPRA, DETALLE_VENTA, DETALLE_PRODUCCION) y la cabecera del módulo de producción (PRODUCCION).


```
SQL*Plus - Diagnostico del respaldo importado (1/2)

==== TABLAS DEL MODELO PRESENTES / FALTANTES TRAS LA IMPORTACIÓN ====

TABLA                                ESTADO
-----
CLIENTE                             OK
COMPRA                              OK
MATERIA_PRIMA                       OK
PRODUCTO                            OK
PROVEEDOR                           OK
VENTA                                OK
CATEGORIA                           FALTA
DETALLE_COMPRA                      FALTA
DETALLE_PRODUCCION                  FALTA
DETALLE_VENTA                       FALTA
EMPLEADO                            FALTA
PRODUCCION                          FALTA

==== CONTEO DE FILAS IMPORTADAS ====

TABLA                                FILAS
-----
CLIENTE                             100
PROVEEDOR                           5
PRODUCTO                            40
MATERIA_PRIMA                       10
COMPRA                              300
VENTA                                2.500

==== CATEGORÍAS REFERENCIADAS POR PRODUCTO (sin tabla que las resuelva) ====

ID_CATEGORIA  PRODUCTOS
-----
1             8
2             7
3             6
4             7
5             4
6             4
7             4
```

Captura 5. Comparación entre el modelo esperado y lo efectivamente importado, y volumetría de las tablas que sí llegaron.

```
SQL*Plus - Diagnostico del respaldo importado (2/2)

==== CONSECUENCIAS DETECTADAS ====

[1] FK_PRODUCTO_CATEGORIA no pudo crearse -> integridad referencial rota
[2] COMPRA y VENTA son cabeceras huérfanas -> el TOTAL no es verificable
    contra ningún detalle: no hay forma de auditar si el importe es correcto
[3] Las 3 vistas de negocio del proyecto no compilan, porque referencian
    DETALLE_COMPRA, DETALLE_VENTA, PRODUCCION, DETALLE_PRODUCCION y EMPLEADO
[4] El módulo de producción completo no existe en el respaldo

DECISIÓN: aplicar el criterio "si los datos no son consistentes, limpiar y
volver a cargar". Se reconstruye el modelo faltante y se regeneran los
detalles de forma determinista, recalculando las cabeceras.
```

Captura 6. Consecuencias del respaldo parcial y decisión adoptada.

Las consecuencias eran de fondo, no cosméticas:

- **Integridad referencial rota.** Los 40 productos referencian siete categorías que ninguna tabla puede resolver. La restricción quedó sin crear y el modelo, sin protección.
- **Importes no auditables.** COMPRA y VENTA son cabeceras que almacenan un TOTAL. Sin sus tablas de detalle no existe ninguna forma de comprobar si ese total es correcto: el dato es, literalmente, no verificable.
- **Vistas de negocio inutilizables.** Las tres vistas del proyecto (v_gestion_compras, v_gestion_ventas y v_control_produccion) referencian precisamente las tablas ausentes, de modo que ninguna compila.
- **Módulo de producción inexistente.** El tercer proceso de negocio del proyecto no está representado en ningún dato.

4. Fase 2: consistencia de los datos

4.1. Criterio aplicado

El enunciado del trabajo establece el procedimiento a seguir: asegurarse de que los datos sean consistentes y, en caso contrario, limpiar la base de datos y volver a cargar los datos. El diagnóstico de la fase anterior encaja exactamente en ese segundo supuesto, con un matiz importante: no se trataba de datos erróneos que hubiera que corregir, sino de datos ausentes. No había nada que preservar en las tablas faltantes porque, sencillamente, no existían.

La estrategia adoptada fue por tanto conservadora con lo que sí llegó y reconstructiva con lo que faltaba:

- Se conservan íntegras las seis tablas importadas: los 100 clientes, 5 proveedores, 40 productos, 10 materias primas y las 2.800 cabeceras de compra y venta con sus fechas y sus relaciones.
- Se recrean las seis tablas faltantes con su DDL completo: claves primarias, foráneas, restricciones UNIQUE y CHECK, índices de apoyo y secuencias.
- Se cargan los maestros ausentes (7 categorías coherentes con el catálogo real de productos y 12 empleados de planta).
- Se generan los detalles transaccionales de forma determinista, mediante un bloque PL/SQL con semilla fija DBMS_RANDOM.SEED(20260723), de modo que el proceso es reproducible: ejecutarlo de nuevo produce exactamente los mismos datos.
- **Se recalculan las cabeceras.** Este es el punto clave del saneamiento: en lugar de conservar unos totales que ningún detalle respaldaba, cada COMPRA.TOTAL y cada VENTA.TOTAL pasa a ser la suma exacta de sus propias líneas. Cabecera y detalle cuadran al centavo por construcción.

Nota. Los importes originales de las cabeceras no se perdieron por descuido: se sustituyeron deliberadamente. Un total que no puede contrastarse contra ningún detalle no es un dato consistente, es un número. El nuevo valor conserva el orden de magnitud y la estacionalidad de los datos originales y, además, es auditable.

4.2. Reconstrucción del modelo

El script 01_reconstruccion_y_carga.sql resuelve la reconstrucción completa en cinco pasos, y es idempotente: puede ejecutarse varias veces seguidas sin provocar errores, porque comienza eliminando los objetos que va a crear.

Tabla 3. Tablas reconstruidas y volumen de datos generado

Tabla reconstruida	Filas	Contenido
CATEGORIA	7	Clasificación comercial del catálogo
EMPLEADO	12	Nómina de planta con cargo y cédula
DETALLE_COMPRA	927	2 a 4 materias primas por factura de compra
DETALLE_VENTA	5.034	1 a 3 productos por factura de venta
PRODUCCION	400	Lotes fabricados entre enero y mayo de 2026
DETALLE_PRODUCCION	582	Consumo de materia prima por lote

Los datos generados no son aleatorios sin criterio, sino coherentes con el negocio: las líneas de compra se valoran con un costo por kilogramo propio de cada materia prima con una

variación de ± 8 %; las líneas de venta se cobran siempre al precio de catálogo del producto; y el consumo de materia prima de cada lote de producción se determina a partir de la categoría del producto fabricado, de modo que un lote de sales consume sal marina y uno de infusiones consume hierbas.

```
SQL*Plus - Reconstruccion del modelo y ejecucion de los scripts del proyecto (1/2)

PS> sqlplus -S usuario_ile/****@localhost:1521/XEPDB1 @01_reconstruccion_y_carga.sql

-- PASO 0: limpieza idempotente
Procedimiento PL/SQL terminado correctamente.

-- PASO 1: DDL de las 6 tablas faltantes
Tabla creada.          (CATEGORIA)
Tabla creada.          (EMPLEADO)
Tabla creada.          (DETALLE_COMPRA)
Tabla creada.          (DETALLE_VENTA)
Tabla creada.          (PRODUCCION)
Tabla creada.          (DETALLE_PRODUCCION)
Tabla modificada.      x28 (PK, FK, UNIQUE y CHECK)
Índice creado.        x8 (índices de apoyo sobre las claves foráneas)
Secuencia creada.     x4

-- PASO 2: carga de datos maestros
7 filas creadas.       (CATEGORIA)
12 filas creadas.      (EMPLEADO)
Confirmación terminada.

-- PASO 3: generación determinista de los detalles (DBMS_RANDOM.SEED fijo)
DETALLE_COMPRA generado: 927 líneas sobre 300 compras
DETALLE_VENTA generado : 5034 líneas sobre 2500 ventas
PRODUCCION generada    : 400 lotes / 582 líneas de consumo

Procedimiento PL/SQL terminado correctamente.

-- PASO 4: se cierra la FK que había quedado pendiente en la importación
Tabla modificada.

-- PASO 5: estadísticas frescas para el optimizador
Procedimiento PL/SQL terminado correctamente.

=== Reconstruccion completada ===
```

Captura 7. Ejecución del script de reconstrucción: DDL, carga de maestros y generación determinista de los detalles.

```

SQL*Plus - Reconstruccion del modelo y ejecucion de los scripts del proyecto (2/2)

PS> sqlplus -S / as sysdba @"-- Permisosotorgados.sql"
Sesión modificada.
Concesión terminada correctamente.      (GRANT CREATE VIEW)
Directorio creado.                      (dir_respalDOS)
Concesión terminada correctamente.      (READ, WRITE ON DIRECTORY)

PS> sqlplus -S usuario_ile/****@localhost:1521/XEPDB1 @Creacion_de_las_vistas.sql
Vista creada.                          (V_GESTION_COMPRAS)
Vista creada.                          (V_GESTION_VENTAS)
Vista creada.                          (V_CONTROL_PRODUCCION)

```

Captura 8. Ejecución de los dos scripts originales del proyecto: permisos y creación de las tres vistas de negocio, ya compilables.

4.3. Las vistas de negocio

Una vez reconstruido el modelo, los dos scripts originales del proyecto se ejecutaron sin modificación alguna. El de permisos otorga CREATE VIEW al usuario y registra el directorio de respaldos; el de vistas crea las tres vistas de negocio, una por integrante del equipo, que ahora sí encuentran todas las tablas que necesitan.

Tabla 4. Vistas de negocio creadas sobre el modelo saneado

Vista	Tablas que combina	Propósito
v_gestion_compras	compra, proveedor, detalle_compra, materia_prima	Trazabilidad del abastecimiento
v_gestion_ventas	venta, cliente, detalle_venta, producto	Facturación comercial sin exponer datos sensibles del cliente
v_control_produccion	produccion, empleado, producto, detalle_produccion, materia_prima	Control de planta y balance de masa

4.4. Auditoría de consistencia

Para sostener la afirmación de que los datos son consistentes se construyó una batería de 24 chequeos automáticos, agrupados en seis familias. Cada chequeo devuelve el número de filas defectuosas que encuentra; el resultado esperado es cero en todos los casos.

Tabla 5. Familias de chequeos de la auditoría de consistencia

Familia	Nº	Qué verifica
Integridad referencial	7	Que no exista ninguna fila huérfana: cada detalle apunta a una cabecera existente y cada clave foránea resuelve
Complejitud	3	Que ninguna cabecera de compra, venta o producción se haya quedado sin líneas de detalle
Cuadre aritmético	5	Que subtotal = cantidad × precio en cada línea, que el total de cada cabecera sea la suma de sus líneas, y que el precio cobrado coincida con el de catálogo
Unicidad	4	Que no haya cédulas, RUC ni nombres de producto duplicados, ni un mismo producto repetido dentro de una factura
Dominio	3	Que no existan importes ni cantidades negativas o nulas, y que cédulas y RUC tengan la longitud legal
Coherencia temporal	2	Que ninguna transacción tenga fecha futura ni caiga fuera del ejercicio

El resultado es 24 chequeos superados sobre 24. El esquema queda con 12 claves primarias, 11 claves foráneas, 5 restricciones de unicidad y 59 restricciones CHECK y NOT NULL activas, sin ningún objeto inválido.

```

SQL*Plus - Auditoria de consistencia (02_auditoria_consistencia.sql) (1/2)

#####
#  AUDITORIA DE CONSISTENCIA - ESQUEMA USUARIO_ILE      #
#####

GRUPO          CHEQUEO                                     MALAS VEREDICTO
-----
Referencial    PRODUCTO con categoria inexistente          0 OK
Referencial    COMPRA con proveedor inexistente                        0 OK
Referencial    VENTA con cliente inexistente                          0 OK
Referencial    DETALLE_COMPRA con compra o materia inexistente        0 OK
Referencial    DETALLE_VENTA con venta o producto inexistente         0 OK
Referencial    PRODUCCION con empleado o producto inexistente         0 OK
Referencial    DETALLE_PRODUCCION con lote o materia inexistente       0 OK
Completitud    COMPRA sin ninguna linea de detalle                    0 OK
Completitud    VENTA sin ninguna linea de detalle                      0 OK
Completitud    PRODUCCION sin consumo de materia prima                 0 OK
Aritmetica     DETALLE_COMPRA: subtotal <> cantidad * precio          0 OK
Aritmetica     DETALLE_VENTA: subtotal <> cantidad * precio            0 OK
Aritmetica     COMPRA.total <> suma de sus detalles                    0 OK
Aritmetica     VENTA.total <> suma de sus detalles                     0 OK
Aritmetica     DETALLE_VENTA: precio cobrado <> precio de catalogo    0 OK
Unicidad       CEDULA duplicada en CLIENTE                             0 OK
Unicidad       RUC duplicado en PROVEEDOR                              0 OK
Unicidad       NOMBRE duplicado en PRODUCTO                           0 OK
Unicidad       Producto repetido dentro de una misma VENTA            0 OK
Dominio        Importes o cantidades <= 0 en cualquier tabla          0 OK
Dominio        CEDULA de cliente con longitud <> 10                   0 OK
Dominio        RUC de proveedor con longitud <> 13                     0 OK
Temporal       Transacciones con fecha futura                          0 OK
Temporal       Transacciones fuera del ejercicio 2026                  0 OK

==== VOLUMETRIA FINAL DEL ESQUEMA ====

```

Captura 9. Resultado de los 24 chequeos de consistencia: todos en OK.

```
SQL*Plus - Auditoria de consistencia (02_auditoria_consistencia.sql) (2/2)

TABLA                                FILAS
-----
CATEGORIA                            7
CLIENTE                             100
COMPRA                               300
DETALLE_COMPRA                       927
DETALLE_PRODUCCION                   582
DETALLE_VENTA                        5.034
EMPLEADO                             12
MATERIA_PRIMA                        10
PRODUCCION                           400
PRODUCTO                             40
PROVEEDOR                             5
VENTA                                2.500

==== OBJETOS INVALIDOS (esperado: ninguno) ====

==== RESTRICCIONES ACTIVAS ====

TIPO                                CANTIDAD
-----
CHECK / NOT NULL                     59
FOREIGN KEY                           11
PRIMARY KEY                           12
UNIQUE                                5
```

Captura 10. Volumetría final del esquema y restricciones de integridad activas.

5. Fase 3: método de migración a NoSQL

5.1. Alternativas evaluadas

Migrar un esquema relacional a MongoDB admite varias estrategias, y la elección condiciona por completo el resultado. Se evaluaron cuatro:

Tabla 6. Alternativas de migración evaluadas

Método	En qué consiste	Valoración
Exportación plana a CSV/JSON e importación con mongoimport	Volcar cada tabla y cargarla como una colección	Descartado. Es rápido pero reproduce el modelo relacional dentro de MongoDB: obliga a resolver los JOIN a mano con \$lookup y desaprovecha por completo el modelo documental
Herramienta comercial de migración	Un producto que automatiza el traspaso tabla a colección	Descartado. Añade una dependencia externa y sigue siendo una traducción literal: la decisión de diseño la toma la herramienta, no el diseñador
Replicación mediante captura de cambios (CDC)	Sincronización continua de Oracle a MongoDB	Descartado. Es la respuesta correcta para una migración en producción sin ventana de parada, pero excede el alcance de una migración puntual
ETL orientado a agregados (elegido)	Un proceso propio que extrae con SQL, transforma los resultados en documentos agregados y los carga con el driver nativo	Elegido. Es el único que permite aplicar la metodología query-first: decidir el esquema documental en función de los patrones de acceso del negocio, no de la forma que tenían las tablas

5.2. Metodología aplicada: diseño query-first

El método elegido parte de una premisa que invierte el orden habitual del diseño relacional. En el modelo relacional se normaliza primero y las consultas se resuelven después mediante JOIN; en el modelo documental se identifican primero las consultas y el esquema se diseña a su servicio. La unidad de diseño deja de ser la entidad y pasa a ser el agregado: el conjunto de datos que la aplicación siempre lee junto.

Los patrones de acceso identificados en el negocio de ILE fueron:

- Consultar una factura de venta con todas sus líneas, o el histórico de facturas de un cliente ordenado por fecha.
- Consultar una factura de compra con todas las materias primas adquiridas, o el histórico de compras a un proveedor.
- Consultar un lote de producción con el operario responsable, el producto obtenido y la materia prima consumida.
- Rankings analíticos por categoría de producto, por cliente y por operario de planta.

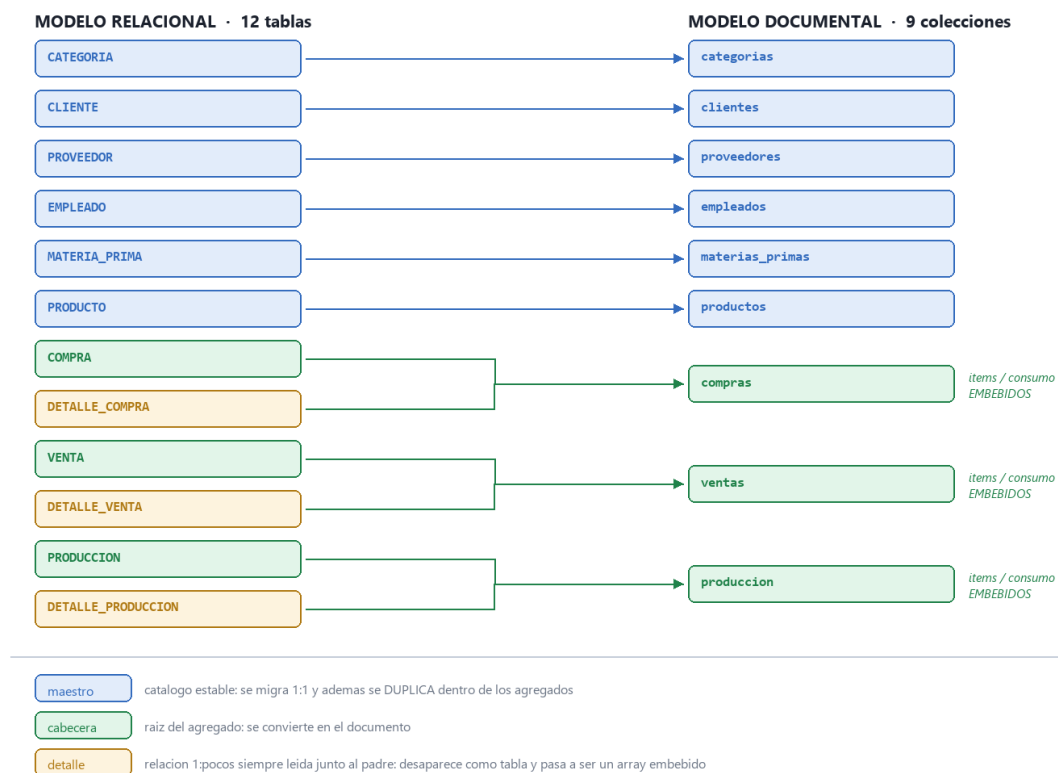
En los tres primeros casos la respuesta es la misma: el detalle nunca se consulta por separado de su cabecera. Es una relación 1:pocos que siempre se lee junta, y por tanto el candidato natural al embebido.

5.3. El modelo documental resultante

La aplicación de estas reglas colapsa las 12 tablas relacionales en 9 colecciones. Las tres tablas de detalle desaparecen como entidades independientes y pasan a ser arrays dentro de su documento padre.

De 12 tablas relacionales a 9 colecciones documentales

Proyecto ILE · Oracle XE 21c (XEPDB1 / usuario_ile) -> MongoDB 8.3 (ile_nosql)



Captura 11. Correspondencia entre el modelo relacional de origen y el modelo documental de destino.

Las reglas concretas que se aplicaron fueron tres:

- **Embeber.** Las líneas de detalle se convierten en un array dentro del documento de la cabecera: items en ventas y compras, consumo en producción. Una factura completa se recupera con una única lectura, sin JOIN.
- **Duplicar el subconjunto estable del maestro.** Dentro de cada línea de venta no se guarda solo el identificador del producto, sino también su nombre y su categoría. Esa duplicación controlada es intencionada: elimina la necesidad de consultar el catálogo para mostrar una factura y, además, congela el dato tal como estaba en el momento de la transacción.
- **Referenciar y conservar los maestros.** Las seis colecciones de catálogo se mantienen completas y siguen siendo la fuente de verdad para el mantenimiento: es donde se actualiza un precio o se da de alta un cliente.

El resultado es un documento autocontenido. La siguiente captura muestra una factura de venta real tal como quedó almacenada: el cliente embebido, dos líneas con su producto y su categoría, y los campos derivados que el ETL precalcula.

```
mongosh - Modelo documental resultante, indices y almacenamiento (1/3)

===== DOCUMENTO DE EJEMPLO: coleccion 'ventas' =====
{
  _id: 1221,
  fecha: ISODate('2026-05-19T00:00:00.000Z'),
  cliente: {
    id: 10,
    cedula: '1107168114',
    nombres: 'Luis 10',
    apellidos: 'Gómez',
    nombre_completo: 'Luis 10 Gómez',
    direccion: 'Sector 10, Loja, Ecuador',
    telefono: '0990000010'
  },
  items: [
    {
      producto: {
        id: 27,
        nombre: 'Sazonador para Carne 80g',
        categoria: {
          id: 4,
          nombre: 'Alinos y Sazonadores'
        }
      },
      cantidad: 2,
      precio_unitario: Decimal128('1.1'),
      subtotal: Decimal128('2.2')
    },
    {
      producto: {
        id: 11,
        nombre: 'Pimienta Dulce en Grano 50g',
        categoria: {
          id: 2,
          nombre: 'Especias Enteras'
        }
      },
      cantidad: 1,
      precio_unitario: Decimal128('1.35'),
      subtotal: Decimal128('1.35')
    }
  ],
  total: Decimal128('3.55'),
}
```

Captura 12. Documento de la colección ventas: cabecera, cliente e items embebidos.

```
mongosh - Modelo documental resultante, índices y almacenamiento (2/3)

n_items: 2,
unidades_totales: 3
}

===== DOCUMENTO DE EJEMPLO: coleccion 'produccion' =====
{
  _id: 1,
  fecha: ISODate('2026-02-04T00:00:00.000Z'),
  cantidad_producida: 1446,
  empleado: {
    id: 1,
    nombre: 'Jorge Anibal Cueva Jaramillo',
    cargo: 'Jefe de Planta',
    cedula: '1104582301'
  },
  producto: {
    id: 39,
    nombre: 'Bicarbonato de Sodio 150g',
    categoria: {
      id: 7,
      nombre: 'Reposteria y Aditivos'
    }
  },
  consumo: [
    {
      materia: {
        id: 8,
        nombre: 'Canela en Corteza Entera',
        unidad_medida: 'Kilogramos'
      },
      cantidad_utilizada: Decimal128('81.23')
    }
  ],
  mp_consumida_total: Decimal128('81.23'),
  n_materias: 1
}
```

Captura 13. Documento de la colección produccion e índices creados sobre ventas.

Nota. Los importes monetarios se migraron como Decimal128 y no como el double que MongoDB usa por defecto para los números. Decimal128 conserva la aritmética decimal exacta del tipo NUMBER(p,s) de Oracle y evita los errores de redondeo binario en las agregaciones. Gracias a esa decisión las sumas de importes coinciden al centavo entre ambos motores.

5.4. Índices derivados de los patrones de acceso

Cada patrón de acceso identificado se materializa en un índice. Destacan los índices multiclave, que MongoDB construye sobre campos que viven dentro de un array: permiten buscar por producto dentro de las líneas de una factura sin que exista una tabla de líneas.

Tabla 7. Índices creados y patrón de acceso que atiende cada uno

Colección	Índice	Patrón de acceso que resuelve
ventas	{cliente.id:1, fecha:-1}	Facturas de un cliente, ya ordenadas de más reciente a más antigua
ventas	{fecha:-1}	Ventas de un periodo
ventas	{items.producto.id:1}	Multiclave: en qué facturas aparece un producto
ventas	{items.producto.categoria.id:1}	Multiclave: ranking por categoría
compras	{proveedor.id:1, fecha:-1}	Compras a un proveedor
compras	{items.materia.id:1}	Multiclave: trazabilidad de una materia prima
produccion	{empleado.id:1, fecha:-1}	Lotes de un operario
produccion	{consumo.materia.id:1}	Multiclave: consumo por materia prima
maestros	{cedula:1} ``{ruc:1}`` `{nombre:1}	Índices únicos que trasladan las restricciones UNIQUE del modelo relacional

La verificación con `explain()` confirma que el índice compuesto funciona como se esperaba: para recuperar las facturas de un cliente, MongoDB examina 34 claves de índice y devuelve exactamente 34 documentos, sin leer ni un documento de más y sin necesidad de ordenar en memoria.

```
mongosh - Modelo documental resultante, índices y almacenamiento (3/3)

===== INDICES DE LA COLECCION 'ventas' =====
_id_ -> {"_id":1}
ix_ventas_cliente_fecha -> {"cliente.id":1,"fecha":-1}
ix_ventas_fecha -> {"fecha":-1}
ix_ventas_producto -> {"items.producto.id":1}
ix_ventas_categoria -> {"items.producto.categoria.id":1}

===== TAMANO DE ALMACENAMIENTO POR COLECCION =====
categorias          7 docs          0.8 KB datos          4.0 KB en disco
clientes            100 docs         17.9 KB datos          4.0 KB en disco
compras             300 docs        231.1 KB datos          4.0 KB en disco
empleados           12 docs          1.6 KB datos          4.0 KB en disco
materias_primas      10 docs          1.0 KB datos          4.0 KB en disco
produccion          400 docs        212.9 KB datos          4.0 KB en disco
productos           40 docs          7.5 KB datos          4.0 KB en disco
proveedores          5 docs          0.6 KB datos          4.0 KB en disco
ventas              2500 docs       1668.9 KB datos          4.0 KB en disco
TOTAL                2142.1 KB datos        36.0 KB en disco

===== PATRON DE ACCESO CRITICO: facturas de un cliente =====
Indice utilizado : ix_ventas_cliente_fecha
Documentos leidos: 34
Documentos devuel: 34
Claves examinadas: 34
Tiempo (ms)      : 0
```

Captura 14. Almacenamiento por colección y plan de ejecución del patrón de acceso crítico, que confirma el uso del índice compuesto.

5.5. Ejecución del ETL

El proceso está implementado en el script 03_etl_oracle_a_mongo.py y se organiza en seis fases. La transformación se apoya en una técnica sencilla y eficiente: la consulta SQL de extracción une la cabecera con su detalle y devuelve las filas ordenadas por el identificador de la cabecera, de modo que el ETL puede recorrer el cursor una sola vez agrupando las filas consecutivas que pertenecen al mismo documento. No hace falta cargar nada en memoria más allá del documento en construcción.

Consulta de extracción del agregado «venta»

```
SELECT v.id_venta, v.fecha, c.id_cliente, c.nombres, ...,
       d.cantidad, d.precio_unitario, d.subtotal,
       p.id_producto, p.nombre, cat.id_categoria, cat.nombre
FROM venta v
JOIN cliente c ON c.id_cliente = v.id_cliente
JOIN detalle_venta d ON d.id_venta = v.id_venta
JOIN producto p ON p.id_producto = d.id_producto
JOIN categoria cat ON cat.id_categoria = p.id_categoria
ORDER BY v.id_venta, d.id_detalle_venta
```

Las 5.034 filas que devuelve esa consulta se colapsan en 2.500 documentos. La carga se realiza por lotes de 500 documentos con `insert_many`, y el proceso completo —incluidas la creación de los índices y las pruebas de verificación— tarda un segundo.

```

Python - ETL Oracle a MongoDB (03_etl_oracle_a_mongo.py) (1/2)

=====
ETL ORACLE -> MONGODB | PROYECTO ILE | UTPL
=====
Origen : Oracle XE 21c localhost:1521/XEPDB1 (esquema usuario_ile)
Destino : MongoDB 8.3 mongodb://localhost:27017 (base ile_nosql)

=====
FASE 1 - COLECCIONES MAESTRAS (migracion 1:1 de catalogos)
=====
-> categorias : 7 documentos
-> clientes : 100 documentos
-> proveedores : 5 documentos
-> empleados : 12 documentos
-> materias_primas : 10 documentos
-> productos : 40 documentos (categoria embebida)

=====
FASE 2 - AGREGADO VENTA (cabecera + cliente + items embebidos)
=====
-> 5034 filas relacionales colapsadas en 2500 documentos

=====
FASE 3 - AGREGADO COMPRA (cabecera + proveedor + items embebidos)
=====
-> 927 filas relacionales colapsadas en 300 documentos

=====
FASE 4 - AGREGADO PRODUCCION (lote + empleado + consumo embebido)
=====
-> 582 filas relacionales colapsadas en 400 documentos

=====
FASE 5 - INDICES (uno por cada patron de acceso identificado)
=====
-> ventas {cliente.id:1, fecha:-1} -> facturas de un cliente
-> ventas {fecha:-1} -> ventas por periodo
-> ventas {items.producto.id:1} -> indice multiclave sobre el array
-> ventas {items.producto.categoria.id:1} -> ranking por categoria
-> compras {proveedor.id:1, fecha:-1} -> compras a un proveedor
-> compras {items.materia.id:1} -> trazabilidad de materia prima
-> produccion {empleado.id:1, fecha:-1} -> lotes por operario

```

Captura 15. Ejecución del ETL: colecciones maestras y construcción de los tres agregados transaccionales.

5.6. Verificación de la migración

Una migración no termina cuando los datos se han copiado, sino cuando se ha demostrado que se copiaron bien. El propio ETL incorpora una fase final de verificación que interroga a ambos motores y compara sus respuestas en 15 pruebas, de tres tipos: conteo de entidades, conteo de líneas embebidas y cuadre de importes.

Tabla 8. Pruebas de verificación posteriores a la migración

Tipo de prueba	Qué compara	Resultado
Conteo de entidades (9 pruebas)	Filas de cada tabla frente a documentos de su colección	OK
Conteo de líneas embebidas (3 pruebas)	Filas de cada tabla de detalle frente a la suma de los elementos de los arrays correspondientes	OK
Cuadre de importes (2 pruebas)	SUM(total) de ventas y de compras en Oracle frente a la agregación equivalente en MongoDB, al centavo	OK
Cuadre de unidades (1 prueba)	Unidades producidas en ambos sistemas	OK

```

Python - ETL Oracle a MongoDB (03_etl_oracle_a_mongo.py) (2/2)

-> produccion {producto.id:1}          -> lotes de un producto
-> produccion {consumo.materia.id:1}    -> consumo por materia prima
-> maestros    indices unicos sobre cedula / ruc / nombre de producto

=====
FASE 6 - VERIFICACION DE LA MIGRACION (Oracle vs MongoDB)
=====

PRUEBA                                ORACLE      MONGODB     RESULTADO
-----
conteo cliente / clientes              100          100    OK
conteo proveedor / proveedores          5             5    OK
conteo empleado / empleados            12            12    OK
conteo materia_prima / materias_primas 10             10    OK
conteo categoria / categorias           7             7    OK
conteo producto / productos             40            40    OK
conteo venta / ventas                  2500          2500    OK
conteo compra / compras                 300            300    OK
conteo produccion / produccion           400            400    OK
lineas detalle_venta -> ventas.items     5034          5034    OK
lineas detalle_compra -> compras.items    927            927    OK
lineas detalle_produccion -> consumo      582            582    OK
importe total de ventas                  29036.3       29036.30    OK
importe total de compras                 3614446.89    3614446.89    OK
unidades producidas                     442098        442098    OK
-----
15/15 pruebas superadas

=====
RESUMEN
=====

categorias          7 documentos
clientes            100 documentos
compras              300 documentos
empleados            12 documentos
materias_primas      10 documentos
produccion           400 documentos
productos            40 documentos
proveedores           5 documentos
ventas              2500 documentos
TOTAL               3374 documentos

Tiempo total: 0.9 s
Estado       : MIGRACION CORRECTA

```

Captura 16. Fase de verificación del ETL: 15 de 15 pruebas superadas, con coincidencia exacta de importes.

5.7. Comparativa de almacenamiento

El modelo documental resulta considerablemente más compacto, pese a duplicar deliberadamente el nombre del producto y su categoría en cada línea. La razón es doble: MongoDB comprime los datos en disco con snappy de forma predeterminada, y el modelo documental necesita muchos menos índices porque no hay claves foráneas que indexar.

Tabla 9. Espacio ocupado por cada modelo

Motor	Datos	Índices	Total
Oracle (12 tablas + 28 índices)	960 KB	2.304 KB	3.264 KB
MongoDB (9 colecciones + 22 índices)	660 KB	544 KB	1.204 KB

Advertencia metodológica. La comparación debe leerse con cautela. Oracle reserva el espacio en extensiones de 64 KB como mínimo, de modo que las tablas pequeñas ocupan más de lo que sus datos necesitan. Aun así, la diferencia en índices —2.304 KB frente a 544 KB— es estructural y no un artefacto de medición: refleja que el modelo documental no necesita indexar claves foráneas que ya no existen.

6. Fase 4: consultas complejas en ambos motores

Para validar que la migración preserve la capacidad analítica del sistema se diseñaron tres consultas complejas y se implementaron dos veces: en SQL sobre el modelo relacional y con el Aggregation Framework sobre el modelo documental. Las tres combinan agrupaciones, subconsultas o etapas anidadas y funciones de ventana.

Tabla 10. Correspondencia entre construcciones SQL y etapas de agregación

Elemento SQL	Equivalente en MongoDB
JOIN	No existe: los datos ya vienen embebidos en el documento
Tabla de detalle	`\$unwind` sobre el array de items
GROUP BY	\$group
COUNT(DISTINCT x)	\$addToSet` seguido de `\$size
WHERE` / `HAVING	\$match
ORDER BY` / `FETCH FIRST	\$sort` / `\$limit
RANK() OVER (...)	\$setWindowFields` con `rank
RATIO_TO_REPORT()	`\$setWindowFields` con ventana global
ROW_NUMBER() = 1	\$sort` seguido de `\$group` con `first

6.1. Consulta 1: rentabilidad por categoría de producto

Objetivo: determinar qué categorías sostienen el negocio, midiendo para cada una el número de facturas en que aparece, las unidades vendidas, el ingreso generado, el ticket medio por línea y su participación porcentual sobre el ingreso total.

En Oracle requiere unir cuatro tablas y aplicar dos funciones analíticas. En MongoDB parte de una sola colección, porque la categoría viaja dentro de cada línea de la factura; basta con desplegar el array y agrupar.

Oracle · SQL

```
SELECT cat.nombre, COUNT(DISTINCT v.id_venta), SUM(dv.cantidad),
       SUM(dv.subtotal), ROUND(AVG(dv.subtotal),2),
       ROUND(RATIO_TO_REPORT(SUM(dv.subtotal)) OVER () * 100, 2),
       RANK() OVER (ORDER BY SUM(dv.subtotal) DESC)
FROM venta v JOIN detalle_venta dv ON dv.id_venta = v.id_venta
      JOIN producto p      ON p.id_producto = dv.id_producto
      JOIN categoria cat    ON cat.id_categoria = p.id_categoria
GROUP BY cat.nombre ORDER BY 4 DESC
```

MongoDB · Aggregation Pipeline

```
db.ventas.aggregate([
  { $unwind: "$items" },
  { $group: { _id: "$items.producto.categoria.nombre",
    facturas: { $addToSet: "$_id" },
    unidades: { $sum: "$items.cantidad" },
    ingreso: { $sum: "$items.subtotal" },
    lineas: { $sum: 1 } } },
  { $setWindowFields: { sortBy: { ingreso: -1 },
    output: { total_global: { $sum: "$ingreso",
      window: { documents: ["unbounded","unbounded"] } },
    puesto: { $rank: {} } } } },
  { $sort: { ingreso: -1 } } ])
```

Resultado: las Especias Molidas encabezan el ranking con 6.334,45 USD y el 21,82 % del ingreso. El dato más interesante es el de las Sales Condimentadas: son terceras en ingreso

pese a aparecer en la mitad de facturas que la categoría líder, porque su ticket por línea (10,02 USD) duplica al del resto. Ambos motores devuelven las siete filas idénticas.

6.2. Consulta 2: top 10 de clientes y su producto estrella

Objetivo: identificar los diez mejores clientes por facturación y, para cada uno, el producto que más unidades le ha comprado. Es una consulta con dos niveles de agregación: primero por cliente y producto, y después por cliente quedándose únicamente con el producto más comprado.

En Oracle se resuelve con dos expresiones de tabla común y una función ROW_NUMBER() particionada por cliente, filtrando por rn = 1. En MongoDB el equivalente idiomático es un doble \$group con un \$sort intermedio: tras ordenar por unidades descendentes, el operador \$first del segundo \$group captura precisamente la primera fila de cada partición.

Oracle · SQL (fragmento)

```
preferencia AS (  
  SELECT id_cliente, producto, unidades,  
         ROW_NUMBER() OVER (PARTITION BY id_cliente  
                           ORDER BY unidades DESC, producto) AS rn  
  FROM (...))  
SELECT ... FROM resumen_cliente r  
JOIN preferencia pr ON pr.id_cliente = r.id_cliente AND pr.rn = 1  
ORDER BY r.total_facturado DESC FETCH FIRST 10 ROWS ONLY
```

MongoDB · Aggregation Pipeline (fragmento)

```
{ $sort: { "_id.cliente": 1, unidades: -1, "_id.producto": 1 } },  
{ $group: { "_id": "$_id.cliente",  
  total_facturado: { $sum: "$importe" },  
  producto_estrella: { $first: "$_id.producto" },  
  uds: { $first: "$unidades" } } },  
{ $sort: { total_facturado: -1 } }, { $limit: 10 }
```

Resultado: encabeza la lista Juan 8 Maldonado con 488,30 USD en 38 facturas. Las diez filas coinciden en ambos motores, salvo un centavo en una celda que se analiza en el apartado 6.4.

6.3. Consulta 3: eficiencia de planta por mes y operario

Objetivo: medir la eficiencia productiva como kilogramos de materia prima consumidos por unidad fabricada, agrupando por mes y operario, y quedarse con los tres más eficientes de cada mes.

Esta consulta ilustra mejor que ninguna la diferencia entre ambos modelos. En el modelo relacional existe una trampa: si se une directamente PRODUCCION con DETALLE_PRODUCCION, la columna cantidad_producida se repite tantas veces como materias primas tenga el lote, y al sumarla el indicador sale inflado. Es el problema conocido como fan-out, y obliga a pre-agregar el detalle en una CTE antes de unirlo con la cabecera.

Oracle · SQL (fragmento)

```
WITH lote AS (  
  SELECT pr.id_produccion, pr.fecha, pr.id_empleado, pr.cantidad_producida,  
         (SELECT SUM(d.cantidad_utilizada) FROM detalle_produccion d  
          WHERE d.id_produccion = pr.id_produccion) AS kg_mp  
  FROM produccion pr)  
SELECT ... RANK() OVER (PARTITION BY mes ORDER BY kg_x_unidad) AS puesto  
FROM metrica ... WHERE puesto <= 3
```

En el modelo documental ese riesgo sencillamente no existe. El consumo total del lote ya vive dentro del propio documento, en el campo `mp_consumida_total` que el ETL precalculó durante la migración. No hay nada que unir y, por tanto, nada que se pueda duplicar.

MongoDB · Aggregation Pipeline (fragmento)

```
db.produccion.aggregate([
  { $group: { _id: { mes: { $dateToString: { format: "%Y-%m",
    date: "$fecha" } } },
    empleado: "$empleado.id" },
    unidades: { $sum: "$cantidad_producida" },
    kg_mp: { $sum: "$mp_consumida_total" } } },
  { $setWindowFields: { partitionBy: "$_id.mes",
    sortBy: { kg_x_unidad: 1 },
    output: { puesto: { $rank: {} } } } },
  { $match: { puesto: { $lte: 3 } } } ])
```

Resultado: 15 filas, tres por cada mes de enero a mayo. La mejor marca del periodo la firma Rosa Elena Sarango Quezada en febrero, con 0,0739 kg de materia prima por unidad producida sobre 11 lotes. Los quince registros coinciden exactamente en ambos motores.

6.4. Comparación de los resultados

Las tres consultas se ejecutaron consecutivamente en cada motor. Las siguientes capturas muestran las salidas completas, listas para contrastar fila a fila.

SQL*Plus - Consultas complejas sobre el modelo relacional (1/2)

```
#####
# CONSULTA 1 - RENTABILIDAD POR CATEGORIA DE PRODUCTO      #
# 4 tablas + GROUP BY + COUNT DISTINCT + 2 funciones      #
# analiticas (RATIO_TO_REPORT y RANK)                      #
#####
```

CATEGORIA	FACTURAS	UNIDADES	INGRESO	TICKET_LINEA	PCT_TOTAL	PUESTO
Espicias Molidas	937	3.199	6.334,45	6,00	21,82	1
Alinos y Sazonadores	786	2.620	5.274,40	6,04	18,16	2
Sales Condimentadas	488	1.558	5.188,10	10,02	17,87	3
Espicias Enteras	786	2.592	4.631,70	5,31	15,95	4
Hierbas Aromaticas	712	2.389	3.670,50	4,74	12,64	5
Infusiones y Bebidas	450	1.437	2.289,30	4,80	7,88	6
Reposteria y Aditivos	448	1.399	1.647,85	3,55	5,68	7

```
#####
# CONSULTA 2 - TOP 10 CLIENTES Y SU PRODUCTO ESTRELLA      #
# 2 CTE + ROW_NUMBER particionado + JOIN entre CTE          #
# (el producto mas comprado por cada cliente)               #
#####
```

CLIENTE	CEDULA	FACTURAS	TOTAL_FACTURADO	TICKET_PROMEDIO	ULTIMA_COMPR	PRODUCTO_ESTRELLA	UDS
Juan 8 Maldonado	1106224361	38	488,30	12,85	29/05/2026	Canela en Rama 50g	16
Juan 16 Gómez	1107719003	33	461,95	14,00	26/05/2026	Albahaca Deshidratada 40g	22
María 31 Gómez	1107596844	35	438,25	12,52	29/05/2026	Aliño Completo ILE 200g	17
Ana 21 Pérez	1108789499	34	429,25	12,63	29/05/2026	Jengibre Molido 60g	22
María 51 Pérez	1108233346	35	423,80	12,11	29/05/2026	Cúrcuma en Polvo 70g	20
Juan 64 Gómez	1103078297	31	411,35	13,27	29/05/2026	Ajo en Polvo ILE 80g	12
Luis 10 Gómez	1107168114	38	396,35	10,43	28/05/2026	Paprika Ahumada 80g	17
Ana 73 Gómez	1107504680	29	391,05	13,48	18/05/2026	Infusión de Menta 25 sobres	14
Ana 5 Maldonado	1100997733	29	379,95	13,10	21/05/2026	Sal Ahumada BBQ 300g	19
Juan 48 Pérez	1109277178	28	369,60	13,20	28/05/2026	Canela Molida 50g	17

Captura 17. Oracle · consultas 1 y 2 sobre el modelo relacional.

SQL*Plus - Consultas complejas sobre el modelo relacional (2/2)

```
#####
# CONSULTA 3 - EFICIENCIA DE PLANTA POR MES Y OPERARIO      #
# Top 3 de cada mes por consumo de materia prima por        #
# unidad producida. CTE previa para evitar el fan-out del    #
# JOIN 1:N con detalle_produccion + RANK particionado        #
#####
```

MES	EMPLEADO	CARGO	LOTES	UNIDADES	KG_MP	KG_X_UNIDAD	PUESTO
2026-01	Diana Carolina Vega Montoya	Tecnica de Control Calidad	5	6.458	540,70	0,0837	1
2026-01	Rosa Elena Sarango Quezada	Operadora de Envasado	8	8.758	747,92	0,0854	2
2026-01	Maria Fernanda Ordóñez Loaiza	Supervisora de Producción	3	2.403	205,46	0,0855	3
2026-02	Rosa Elena Sarango Quezada	Operadora de Envasado	11	11.011	814,11	0,0739	1
2026-02	Diana Carolina Vega Montoya	Tecnica de Control Calidad	4	3.268	251,60	0,0770	2
2026-02	Nube Patricia Armijos Torres	Operadora de Envasado	5	5.283	429,57	0,0813	3
2026-03	Maria Fernanda Ordóñez Loaiza	Supervisora de Producción	4	5.383	465,60	0,0865	1
2026-03	Diana Carolina Vega Montoya	Tecnica de Control Calidad	6	7.388	657,46	0,0890	2
2026-03	Carlos Andres Tandazo Riofrio	Operador de Molienda	14	14.358	1.282,31	0,0893	3
2026-04	Segundo Manuel Japon Curipoma	Operador de Secado	9	9.209	737,13	0,0800	1
2026-04	Silvia Marlene Camacho Ruiz	Operadora de Etiquetado	7	8.666	726,16	0,0838	2
2026-04	Maria Fernanda Ordóñez Loaiza	Supervisora de Producción	9	9.954	845,27	0,0849	3
2026-05	Jorge Anibal Cueva Jaramillo	Jefe de Planta	11	11.023	923,29	0,0838	1
2026-05	Maria Fernanda Ordóñez Loaiza	Supervisora de Producción	7	8.346	721,79	0,0865	2
2026-05	Rosa Elena Sarango Quezada	Operadora de Envasado	3	4.880	425,18	0,0871	3

Captura 18. Oracle · consulta 3 sobre el modelo relacional.

```
mongosh - Las mismas consultas sobre el modelo documental (1/2)

#####
# CONSULTA 1 - RENTABILIDAD POR CATEGORIA DE PRODUCTO #
# $unwind + $group + $addToSet + $setWindowFields #
# ($rank y ventana global para el % de participacion) #
#####

CATEGORIA          FACTURAS  UNIDADES  INGRESO  TICKET_LINEA  PCT_TOTAL  PUESTO
-----
Especias Molidas    937      3.199     6.334,45    6,00         21,82      1
Alinos y Sazonadores 786      2.620     5.274,40    6,04         18,16      2
Sales Condimentadas 488      1.558     5.188,10    10,02        17,87      3
Especias Enteras    786      2.592     4.631,70    5,31         15,95      4
Hierbas Aromaticas  712      2.389     3.670,50    4,74         12,64      5
Infusiones y Bebidas 450      1.437     2.289,30    4,80         7,88       6
Reposteria y Aditivos 448      1.399     1.647,85    3,55         5,68       7
-----
7 fila(s)

#####
# CONSULTA 2 - TOP 10 CLIENTES Y SU PRODUCTO ESTRELLA #
# doble $group + $sort intermedio + $first #
# (equivalente a ROW_NUMBER() OVER (PARTITION BY...) = 1)#
#####

CLIENTE            CEDULA      FACTURAS  TOTAL_FACT.  TICKET_PROM.  ULTIMA_COMPRA  PRODUCTO_ESTRELLA  UDS
-----
Juan 8 Maldonado    1106224361  38        488,30       12,85         29/05/2026         Canela en Rama 50g  16
Juan 16 Gómez       1107719003  33        461,95       14,00         26/05/2026         Albahaca Deshidratada 40g  22
María 31 Gómez      1107596844  35        438,25       12,52         29/05/2026         Aliño Completo ILE 200g  17
Ana 21 Pérez        1108789499  34        429,25       12,62         29/05/2026         Jengibre Molido 60g      22
María 51 Pérez      1108233346  35        423,80       12,11         29/05/2026         Cúrcuma en Polvo 70g     20
Juan 64 Gómez       1103078297  31        411,35       13,27         29/05/2026         Ajo en Polvo ILE 80g     12
Luis 10 Gómez       1107168114  38        396,35       10,43         28/05/2026         Paprika Ahumada 80g      17
Ana 73 Gómez        1107504680  29        391,05       13,48         18/05/2026         Infusión de Menta 25 sobres  14
Ana 5 Maldonado     1100997733  29        379,95       13,10         21/05/2026         Sal Ahumada BBQ 300g     19
Juan 48 Pérez       1109277178  28        369,60       13,20         28/05/2026         Canela Molida 50g        17
-----
10 fila(s)
```

Captura 19. MongoDB · consultas 1 y 2 sobre el modelo documental.

```
mongosh - Las mismas consultas sobre el modelo documental (2/2)

#####
# CONSULTA 3 - EFICIENCIA DE PLANTA POR MES Y OPERARIO #
# $group por mes/empleado + $setWindowFields con $rank #
# (sin riesgo de fan-out: el consumo ya esta en el documento)#
#####

MES      EMPLEADO          CARGO          LOTES  UNIDADES  KG_MP  KG_X_UNIDAD  PUESTO
-----
2026-01  Diana Carolina Vega Montoya  Tecnica de Control Calidad  5      6.458     540,70    0,0837      1
2026-01  Rosa Elena Sarango Quezada  Operadora de Envasado      8      8.758     747,92    0,0854      2
2026-01  Maria Fernanda Ordonez Loaiza  Supervisora de Produccion  3      2.403     205,46    0,0855      3
2026-02  Rosa Elena Sarango Quezada  Operadora de Envasado     11     11.011     814,11    0,0739      1
2026-02  Diana Carolina Vega Montoya  Tecnica de Control Calidad  4      3.268     251,60    0,0770      2
2026-02  Nube Patricia Armijos Torres  Operadora de Envasado      5      5.283     429,57    0,0813      3
2026-03  Maria Fernanda Ordonez Loaiza  Supervisora de Produccion  4      5.383     465,60    0,0865      1
2026-03  Diana Carolina Vega Montoya  Tecnica de Control Calidad  6      7.388     657,46    0,0890      2
2026-03  Carlos Andres Tandazo Riofrio  Operador de Molienda     14     14.358     1.282,31  0,0893      3
2026-04  Segundo Manuel Japon Curipoma  Operador de Secado        9      9.209     737,13    0,0800      1
2026-04  Silvia Marlene Camacho Ruiz  Operadora de Etiquetado    7      8.666     726,16    0,0838      2
2026-04  Maria Fernanda Ordonez Loaiza  Supervisora de Produccion  9      9.954     845,27    0,0849      3
2026-05  Jorge Anibal Cueva Jaramillo  Jefe de Planta           11     11.023     923,29    0,0838      1
2026-05  Maria Fernanda Ordonez Loaiza  Supervisora de Produccion  7      8.346     721,79    0,0865      2
2026-05  Rosa Elena Sarango Quezada  Operadora de Envasado      3      4.880     425,18    0,0871      3
-----
15 fila(s)
```

Captura 20. MongoDB · consulta 3 sobre el modelo documental.

De las 32 filas devueltas en total y sus más de doscientos valores, todos coinciden salvo uno: en la consulta 2, el ticket promedio del cliente Ana 21 Pérez es 12,63 en Oracle y 12,62 en MongoDB.

La diferencia no es un error de la migración, sino una diferencia de criterio entre los dos motores. El cálculo es 429,25 dividido entre 34 facturas, que da exactamente 12,625: un valor situado justo en la mitad entre dos centavos. Ante ese empate cada motor aplica una regla distinta:

- **Oracle.** La función ROUND() aplica redondeo por mitad hacia arriba (round half away from zero), el criterio aritmético clásico: 12,625 → 12,63.
- **MongoDB.** El operador \$round aplica redondeo bancario (round half to even), que ante un empate elige el dígito par para no sesgar sistemáticamente los agregados al alza: 12,625 → 12,62.

Sobre esta diferencia. Es el único desacuerdo encontrado y aparece únicamente cuando el valor cae exactamente en la mitad, algo que sucede en 1 de las 32 filas. Conviene documentarlo porque, aplicado a millones de operaciones financieras, el criterio de redondeo deja de ser un detalle. Si la coincidencia estricta con Oracle fuera un requisito, bastaría con reemplazar \$round por la expresión equivalente al redondeo por mitad hacia arriba. El hecho de que las sumas de importes —donde no interviene ningún redondeo— coincidan al centavo confirma que los datos migrados son idénticos.

Tabla 11. Contraste de resultados entre ambos motores

Consulta	Filas devueltas	Coincidencia Oracle / MongoDB
1. Rentabilidad por categoría	7	Idéntica en todos los valores
2. Top 10 clientes y producto estrella	10	Idéntica salvo un centavo por redondeo en una celda
3. Eficiencia de planta por mes	15	Idéntica en todos los valores

7. Conclusiones

La verificación de consistencia no es un trámite previo a la migración, sino la parte que determina si la migración tiene sentido. En este caso el respaldo entregado estaba incompleto y, de haberse migrado tal cual, el resultado habría sido una base MongoDB con cabeceras sin líneas y una clave foránea rota: los mismos defectos, solo que en otro motor. Migrar datos inconsistentes no los arregla, los propaga.

La migración a NoSQL no es una traducción, es un rediseño. La tentación de convertir cada tabla en una colección produce un modelo relacional disfrazado, que obliga a simular los JOIN con \$lookup y no aporta ninguna de las ventajas del modelo documental. El método correcto empieza por preguntarse qué consultas debe responder el sistema, y deja que sea esa respuesta la que determine la estructura de almacenamiento. En este trabajo esa decisión redujo 12 tablas a 9 colecciones y eliminó por completo la necesidad de reunir tablas en las consultas de negocio.

La denormalización controlada es una decisión de diseño, no un descuido. Duplicar el nombre y la categoría del producto dentro de cada línea de factura contradice la tercera forma normal, y es precisamente lo que hace que una factura completa se lea de una sola vez. El coste —mantener esa copia actualizada si el catálogo cambia— es asumible porque un nombre de producto es un dato estable, y además tiene un efecto secundario deseable: congela el dato tal como estaba en el momento de la transacción.

Los detalles de tipado importan. Migrar los importes como Decimal128 en lugar de como el double por defecto es lo que permite que las sumas de importes coincidan al centavo entre ambos motores. Y la única discrepancia encontrada en todo el proceso —un centavo en una celda— no vino de los datos, sino de que Oracle y MongoDB resuelven de forma distinta el empate exacto en el redondeo. Son el tipo de detalles que solo aparecen cuando se verifica en serio, comparando resultados en lugar de suponer que coinciden.

Cada modelo conserva sus fortalezas. El modelo relacional impone la integridad desde el motor: 11 claves foráneas y 59 restricciones garantizan que ningún dato incoherente entre en la base. El modelo documental traslada esa responsabilidad a la aplicación y al proceso de carga, y a cambio ofrece lecturas de un solo acceso y un esquema flexible. La consulta 3 lo resume bien: en SQL hubo que protegerse del fan-out con una CTE previa, mientras que en MongoDB el dato agregado ya estaba dentro del documento. Ninguno de los dos modelos es superior en abstracto; son respuestas distintas a preguntas distintas.